

HTML & CSS Assignment 3

Question 1: Social App Icons Row (Flexbox + justify-content)

Create an HTML file with a container div and 4 images of your favorite apps (Instagram, WhatsApp, Zomato, Spotify) as social icons. Use CSS Flexbox to display them in a row with equal spacing using `justify-content`.

social-icons.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Social App Icons</title>
  <style>
    .icon-container {
      display: flex;
      justify-content: space-between; /* equal spacing between icons */
      align-items: center;
      width: 320px;
      padding: 15px;
      border: 1px solid #ddd;
      border-radius: 8px;
    }

    .icon-container img {
      width: 45px;
      height: 45px;
      border-radius: 10px;
    }
  </style>
</head>
<body>

  <div class="icon-container">
    
    
    
    
  </div>

</body>
</html>
```

Explanation: The `.icon-container` is set to `display: flex`, which arranges the four image children in a horizontal row by default. `justify-content: space-between` distributes the extra horizontal space evenly between the icons, pushing the first icon to the start and the last icon to the end, with equal gaps in between. `align-items: center` keeps them vertically centered within the container.

Question 2: Spotify-style Horizontal Playlist Bar (Flexbox + align-items)

Build a horizontal playlist bar like Spotify using Flexbox: add 5 song cards with song name and artist, and use `align-items` to vertically center the text in each card.

playlist-bar.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Spotify Style Playlist Bar</title>
  <style>
    .playlist-bar {
      display: flex;
      gap: 12px;
      overflow-x: auto;
      background-color: #121212;
      padding: 16px;
    }

    .song-card {
      display: flex;
      flex-direction: column;
      align-items: center; /* vertically centers text horizontally within the card */
      justify-content: center;
      min-width: 120px;
      background-color: #1e1e1e;
      color: #ffffff;
      padding: 14px;
      border-radius: 8px;
      font-family: Arial, sans-serif;
      text-align: center;
    }

    .song-name {
      font-size: 14px;
      font-weight: bold;
      margin-bottom: 4px;
    }

    .artist-name {
      font-size: 12px;
      color: #b3b3b3;
    }
  </style>
</head>
<body>

  <div class="playlist-bar">

    <div class="song-card">
      <span class="song-name">Blinding Lights</span>
      <span class="artist-name">The Weeknd</span>
    </div>

    <div class="song-card">
      <span class="song-name">Levitating</span>
      <span class="artist-name">Dua Lipa</span>
    </div>

    <div class="song-card">
      <span class="song-name">Shape of You</span>
      <span class="artist-name">Ed Sheeran</span>
    </div>

    <div class="song-card">
      <span class="song-name">Stay</span>
      <span class="artist-name">Kid Laroi, Bieber</span>
    </div>

  </div>

</body>
</html>
```

```

    <div class="song-card">
      <span class="song-name">Perfect</span>
      <span class="artist-name">Ed Sheeran</span>
    </div>

  </div>

</body>
</html>

```

Explanation: The outer `.playlist-bar` uses `display: flex` (default row direction) to lay out the 5 song cards side by side horizontally, similar to Spotify's scrollable playlist row, with `overflow-x: auto` allowing horizontal scrolling if needed. Each `.song-card` is itself a flex container with `flex-direction: column`, stacking the song name above the artist name, while `align-items: center` vertically/horizontally centers this text block within the card's cross-axis.

Question 3: Flipkart-style Product Grid (CSS Grid, 3 columns × 2 rows)

Design a Flipkart-style product grid: create a section with 6 product cards (use placeholder images and product names), and use CSS Grid to arrange them in 3 columns and 2 rows.

product-grid.html

```

<!DOCTYPE html>
<html>
<head>
  <title>Flipkart Style Product Grid</title>
  <style>
    .product-grid {
      display: grid;
      grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
      grid-template-rows: repeat(2, auto); /* 2 rows */
      gap: 16px;
      padding: 20px;
      font-family: Arial, sans-serif;
    }

    .product-card {
      border: 1px solid #e0e0e0;
      border-radius: 6px;
      padding: 12px;
      text-align: center;
    }

    .product-card img {
      width: 100%;
      border-radius: 4px;
    }

    .product-card p {
      margin-top: 8px;
      font-size: 14px;
      color: #212121;
    }
  </style>
</head>
<body>

  <section class="product-grid">

```

```

    <div class="product-card"><p>Wireless Earbuds</p>
</div>
    <div class="product-card"><p>Smart Watch</p></div>
    <div class="product-card"><p>Bluetooth Speaker</p>
</div>
    <div class="product-card"><p>Laptop Backpack</p>
</div>
    <div class="product-card"><p>Gaming Mouse</p></div>
    <div class="product-card"><p>Power Bank</p></div>
</section>

</body>
</html>

```

Explanation: The `.product-grid` section uses `display: grid` with `grid-template-columns: repeat(3, 1fr)`, creating three equal-width columns, and `grid-template-rows: repeat(2, auto)` for two rows. Since there are exactly 6 product cards, the grid automatically fills them in as 3 columns × 2 rows, with the `gap` property adding consistent spacing between all cards both horizontally and vertically.

Question 4: Making the Product Grid Responsive (Media Queries)

Make your product grid responsive: use `grid-template-columns` to show 3 columns on desktop, 2 columns on tablet, and 1 column on mobile screens.

product-grid-responsive.html

```

<!DOCTYPE html>
<html>
<head>
  <title>Responsive Flipkart Product Grid</title>
  <style>
    .product-grid {
      display: grid;
      grid-template-columns: repeat(3, 1fr); /* Desktop: 3 columns (default) */
      gap: 16px;
      padding: 20px;
      font-family: Arial, sans-serif;
    }

    .product-card {
      border: 1px solid #e0e0e0;
      border-radius: 6px;
      padding: 12px;
      text-align: center;
    }

    .product-card img {
      width: 100%;
      border-radius: 4px;
    }

    .product-card p {
      margin-top: 8px;
      font-size: 14px;
      color: #212121;
    }

    /* Tablet screens: 2 columns */
    @media (max-width: 900px) {

```

```

        .product-grid {
            grid-template-columns: repeat(2, 1fr);
        }
    }

    /* Mobile screens: 1 column */
    @media (max-width: 500px) {
        .product-grid {
            grid-template-columns: 1fr;
        }
    }
</style>
</head>
<body>

    <section class="product-grid">
        <div class="product-card"><p>Wireless Earbuds</p>
    </div>
        <div class="product-card"><p>Smart Watch</p></div>
        <div class="product-card"><p>Bluetooth Speaker</p>
    </div>
        <div class="product-card"><p>Laptop Backpack</p>
    </div>
        <div class="product-card"><p>Gaming Mouse</p></div>
        <div class="product-card"><p>Power Bank</p></div>
    </section>

</body>
</html>

```

Explanation: By default (desktop / wide screens), `grid-template-columns: repeat(3, 1fr)` shows 3 equal columns. A media query with `max-width: 900px` (tablet-sized screens and below) overrides this to `repeat(2, 1fr)`, switching the grid to 2 columns. A second media query with `max-width: 500px` (mobile-sized screens) further overrides it to a single column (`1fr`), stacking all product cards vertically. Because CSS applies the last matching rule, as the screen gets narrower, each media query's breakpoint takes over automatically, making the layout responsive without any JavaScript.